

ISS Tracker Application

Overview and Motivation

The ISS Tracker application allows users to explore real-time trajectory and positional data of the International Space Station (ISS). With increasing interest in space exploration and satellite tracking, accessible tools that visualize and interpret satellite data are extremely important. The primary motivation behind this project was to build a user-friendly platform that allows anyone with a basic technical background to query and understand complex orbital data through a straightforward web-based API. By leveraging NASA's publicly available ISS trajectory data, this application serves as a functional demonstration of modern software practices, integrating containerization, data persistence, and web APIs to deliver reliable, precise information to end users.

Application Functionality

The ISS Tracker is written in Python and leverages Flask to create a web-based Representational State Transfer (RESTful) API. Users can query this API through simple HTTP requests to retrieve various pieces of ISS data, including:

- A complete list of orbital state vectors, known as "epochs," each containing positional (X, Y, Z) and velocity data (X_DOT, Y_DOT, Z_DOT).
- Specific details such as instantaneous speed and geolocation for individual epochs.
- The current real-time position of the ISS which is calculated from the latest available epoch data.

When users start the application, Docker Compose deploys two containerized services:

1. **Flask Web API Container:** This service manages user requests, processes ISS trajectory data, performs calculations, and interfaces with the Redis container.
2. **Redis Database Container:** This container acts as a persistent storage solution. Persistent storage is a way to store data that remains accessible even after a device is powered off. It stores the fetched ISS trajectory data, allowing the Flask API to retrieve information effectively.

Upon initialization, the application checks the Redis database for previously stored data. If data is not present, the Flask app automatically fetches the ISS trajectory data from NASA's external data source and caches it within Redis, improving response times for subsequent user queries.

Important Components

- **Docker ([Dockerfile](#)):** Builds the Python environment, copies all necessary files, and installs dependencies.
- **Docker Compose ([docker-compose.yml](#)):** Ensures smooth coordination between the Flask and Redis containers, allowing seamless deployment.
- **Redis Database:** Provides persistent, quick-access storage for the ISS dataset, allowing the Flask API to respond to user queries.
- **Flask Application ([iss_tracker.py](#)):** Manages routes, data fetching, calculations (speed and geographic position), and API endpoint logic.
 - [calculate_speed](#): Computes instantaneous ISS speed from velocity components.
 - [calculate_location](#): Converts Cartesian coordinates (X, Y, Z) into geographic latitude, longitude, and altitude. Converts these coordinates to human-readable locations.
 - [find_closest_epoch](#): Determines the epoch closest to the current system time.
 - [doy_to_isoformat](#): Converts timestamps from day-of-year format to standard ISO format timestamps.
 - [/epochs](#): Returns the entire dataset or paginated data.
 - [/epochs/<epoch>](#): Provides state vectors (position and velocity) for a given epoch.
 - [/epochs/<epoch>/speed](#): Calculates and returns instantaneous speed for a given epoch.
 - [/epochs/<epoch>/location](#): Returns latitude, longitude, altitude, and geoposition for a specified epoch.
 - [/now](#): Determines and returns information (location, speed) for the epoch closest to the current time.
- **Unit Testing ([test_iss_tracker.py](#)):** A set of tests that ensures reliability and correctness by systematically validating each component and API endpoint.
 - [test_get_epochs](#): Tests retrieval of entire dataset and query parameter functionality.
 - [test_get_epoch](#): Tests correct data retrieval for specific epochs.
 - [test_get_epoch_speed](#): Validates instantaneous speed calculation accuracy.
 - [test_get_epoch_location](#): Checks correct geographic conversions and reverse geocoding.
 - [test_get_now](#): Confirms correct functionality of retrieving the nearest epoch to current time.
 - [test_get_invalid_epoch](#): Verifies invalid epoch inputs are handled correctly.
 - [test_calculate_speed](#): Tests the accuracy of speed calculation from velocity components.
 - [test_doy_to_isoformat](#): Tests date conversions from DOY to ISO format.
 - [test_calculate_location](#): Ensures accurate geographic coordinate conversions.

- **test_redis_data_storage:** Verifies that data storage in Redis functions correctly.

Ethical and Professional Responsibilities in Engineering

When developing applications that utilize external data—especially those related to sensitive domains such as space exploration—ethical and professional responsibilities become vital. The ISS Tracker addresses these responsibilities in several integral ways:

- **Data Citation:** Proper citation of data sources demonstrates respect for intellectual property and ensures transparency. This project explicitly cites NASA's trajectory data source, acknowledging the significant efforts and contributions of NASA scientists.
- **Data Quality and Accuracy Verification:** The accuracy of data is critical, particularly when used in contexts involving navigation or scientific analysis. This application emphasizes careful parsing, accurate calculations, and thorough testing to ensure reliability and correctness.
- **Defensive Programming:** The application enforces defensive programming strategies such as thorough input validation, exception handling, and error messaging to prevent misuse and ensure recovery from unexpected errors or “bad input”.
- **Transparency in AI Usage:** AI tools were utilized for certain aspects of the project. All AI-generated code or content is explicitly documented within the application files, respecting intellectual transparency and honesty.

Adhering to these ethical and professional responsibilities ensures that users can confidently rely on the accuracy of the information provided by the ISS Tracker.

Data Source

The ISS trajectory data used by this project is publicly provided by NASA and can be accessed at the following link:

[NASA ISS Trajectory Data](#)

The provided data includes the ISS's timestamped position and velocity vectors, which are used for accurate location and trajectory calculations.

Conclusion

This ISS Tracker application represents a practical integration of modern web technologies, data-handling practices, and clear documentation of professional responsibilities. It serves both as a tool for visualizing ISS movements and as a demonstration of responsible software engineering practices.